DSA Practice - Revision Sheet

Java DSA workspace • auto-generated from your solved files • source files unchanged
Total problems solved across 9 topics: **80**

1. Per-Topic Summary

Topic	Solved	Key Pattern / Technique	Anchor Problem
Arrays	15	Core techniques: linear & binary search, two-pointer (reverse, rota...	Kadane's Algorithm - maximum subarray sum in O(n)
Strings	9	Core techniques: char-by-char traversal, two-pointer palindrome che...	Anagram check using a 26-length frequency array
Bit Manipulation	14	Core techniques: bitmask (1<<i) to get/set/clear/update a bit, AND ...	Get/Set/Clear i-th bit using a (1<<i) bitmask
Divide and Conquer	3	Core technique: split the problem in half, solve recursively, combine	Merge Sort - divide, recursively sort halves, merge (O(n log n))
Matrix	7	Core techniques: 2D traversal (row/col nested loops), diagonal inde...	Staircase Search - O(n+m) search in a sorted matrix
OOP	11	Core concepts: classes & objects, the four pillars (Encapsulation, ...	The 4 pillars + constructors (shallow vs deep copy)
Recursion	14	Core techniques: base case + recursive call, head vs tail recursion...	Fast power x^n by halving the exponent (O(log n))
Sorting	6	Core techniques: comparison sorts - bubble (swap adjacent), selecti...	Insertion Sort - shift each element into its sorted position
Practice	1	Mixed revision folder	Kadane's Algorithm (revision)
TOTAL	**80**		

Thin topics to reinforce: **Practice (1)**, **Divide and Conquer (3)**, and **Sorting (6)** have the fewest solved problems.

2. Topic Details - Concept & Core Logic of Every File

Each entry lists the file, its core concept, and the main logic extracted from your code.

Arrays (15 solved)

Patterns covered: Core techniques: linear & binary search, two-pointer (reverse, rotate via triple-reverse), prefix sums, Kadane's running-sum for max subarray, greedy single-pass (stock, largest), auxiliary left/right-max arrays (trapping rain water), in-place slow/fast pointer (remove duplicates), and next-permutation (find pivot, swap successor, reverse suffix).

Anchor problem: Kadane's Algorithm - maximum subarray sum in O(n)

• *arrayfunction.java*

Concept: Passing arrays to methods; arrays are passed by reference so in-place mutation (adding 5 to each mark) reflects in caller.

```
public static void arr(int marks[]){
    for(int i=0;i&lt;marks.length;i++){
        marks[i]+=5;
    }
}
```

• *linearsearch.java*

Concept: Sequential scan; return index on first match, else -1. O(n).

```
public static int right(int marks[],int key){
    for(int i=0;i&lt;marks.length;i++){
        if(marks[i]==key) return i;
    }
    return -1;
}
```

• *binarysearch.java*

Concept: Search in sorted array by repeatedly halving the range using start/end/mid. O(log n).

```
public static int bs(int marks[],int key){
    int start=0,end=marks.length-1;
    while(start&lt;=end){
        int mid=(start+end)/2;
        if(marks[mid]==key) return mid;
        if(marks[mid]&gt;key) end=mid-1;
        else start=mid+1;
    }
    return -1;
}
```

• *printsubarray.java*

Concept: Generate every contiguous subarray with triple nested loops (start i, end j, print k). Total subarrays = n(n+1)/2. O(n^3).

```
for(int i=0;i&lt;marks.length;i++){
    int start=i;
    for(int j=i;j&lt;marks.length;j++){
        int end=j;
        for(int k=start;k&lt;=end;k++)
            System.out.print(marks[k]);
        ts++;
    }
}
```

• *largestno.java*

Concept: Track running maximum starting from Integer.MIN_VALUE. O(n).

```
int large=Integer.MIN_VALUE;
for(int i=0;i&lt;marks.length;i++){
    if(large&lt;marks[i]) large=marks[i];
}
return large;
```

• *dsatraprainwater.java*

Concept: Auxiliary arrays: precompute leftMax and rightMax for each bar; trapped water at i = min(leftMax,rightMax) - height. O(n) time, O(n) space.

```
leftmax[0]=marks[0];
for(int i=1;i&lt;n;i++) leftmax[i]=Math.max(marks[i],leftmax[i-1]);
rightmax[n-1]=marks[n-1];
for(int i=n-2;i&gt;=0;i--) rightmax[i]=Math.max(marks[i],rightmax[i+1]);
for(int i=0;i&lt;n;i++)
    trappedwater += Math.min(leftmax[i],rightmax[i]) - marks[i];
```

• *dsastockbuyandsell.java*

Concept: Greedy single pass: track min buy price so far, update max profit = max(profit, price-buy). O(n).

```
int maxprofit=0, buyprice=Integer.MAX_VALUE;
for(int i=0;i&lt;prices.length;i++){
    if(buyprice&lt;prices[i])
        maxprofit=Math.max(prices[i]-buyprice,maxprofit);
    else
        buyprice=prices[i];
}
return maxprofit;
```

• *reverseofarray.java*

Concept: Two-pointer swap from both ends moving inward. In-place, O(n).

```
int first=0,last=marks.length-1;
while(first&lt;=last){
    int temp=marks[last];
    marks[last]=marks[first];
    marks[first]=temp;
    first++; last--;
}
```

• *trapwater.java*

Concept: Same trapping-rain-water logic (cleaner n variable). min(leftMax,rightMax)-height summed.

```
for(int i=0;i&lt;n;i++)
    trappedwater += Math.min(leftmax[i],rightmax[i]) - height[i];
return trappedwater;
```

• *prefixsumforsubarray.java*

Concept: Prefix-sum array lets any subarray sum be computed in O(1): sum(i..j)=prefix[j]-prefix[i-1]. Overall O(n^2) for max subarray.

```
prefix[0]=marks[0];
for(int i=1;i&lt;n;i++) prefix[i]=prefix[i-1]+marks[i];
for(int i=0;i&lt;n;i++)
  for(int j=i;j&lt;n;j++){
    currsum = (i==0)? prefix[j] : prefix[j]-prefix[i-1];
    if(currsum&gt;maxsum) maxsum=currsum;
  }
```

• *maxsubarraysumbruteforce.java*

Concept: Running-sum variant of Kadane: add element, update max, reset to 0 when negative. O(n).

```
for(int i=0;i&lt;marks.length;i++){
    currsum += marks[i];
    if(currsum&gt;maxsum) maxsum=currsum;
    if(currsum&lt;0) currsum=0;
}
```

• *rotatearaybykelement.java*

Concept: Rotate right by k using triple reversal: reverse[0..k-1], reverse[k..n-1], reverse whole. O(n), O(1) space.

```
k=k%n;
reverse(arr,0,k-1);
reverse(arr,k,n-1);
reverse(arr,0,n-1);
```

• *removeduplicates.java*

Concept: Two-pointer on sorted array: slow i holds last unique, fast j scans; write when arr[i]!=arr[j]. Returns new length.

```
int i=0;
for(int j=1;j&lt;arr.length;j++){
    if(arr[i]!=arr[j]){
        i++;
        arr[i]=arr[j];
    }
}
return i+1;
```

• *kadanealgo.java*

Concept: Kadane's algorithm: max running sum, reset when negative. Handles the classic max-subarray problem in O(n).

```
int currsum=0, maxsum=Integer.MIN_VALUE;
for(int i=0;i&lt;marks.length;i++){
    currsum+=marks[i];
    maxsum=Math.max(maxsum,currsum);
    if(currsum&lt;0) currsum=0;
}
```

• *nextpermutation.java*

Concept: Next lexicographic permutation: from right find first i with arr[i]<arr[i+1]; find successor j; swap; reverse suffix. O(n).

```
int i=arr.length-2;
while(i&gt;=0 &amp;&amp; arr[i]&gt;=arr[i+1]) i--;
if(i&gt;=0){
    int j=arr.length-1;
    while(arr[j]&lt;=arr[i]) j--;
    swap(arr,i,j);
}
reverse(arr,i+1,arr.length-1);
```

Strings (9 solved)

Patterns covered: Core techniques: char-by-char traversal, two-pointer palindrome check, frequency arrays (int[26]) for anagrams, StringBuilder for efficient building/compression, coordinate tracking for path problems, and lexicographic comparison with compareTo().

Anchor problem: Anagram check using a 26-length frequency array

• *basicstring.java*

Concept: Traverse string in reverse with charAt to print it backwards.

```
for(int i=str.length()-1;i&gt;=0;i--)
    System.out.print(str.charAt(i));
```

• *stringpalindrome.java*

Concept: Two-pointer: compare char i with char (n-1-i) up to the middle.

```
for(int i=0;i&lt;str.length()/2;i++){
    if(str.charAt(i)!=str.charAt(str.length()-i-1)){
        System.out.println("not a palindrome"); return;
    }
}
```

• *shortestpath.java*

Concept: Track x,y displacement from N/S/E/W moves; shortest path = sqrt(x^2+y^2) (straight-line distance).

```
for(int i=0;i&lt;path.length();i++){
    char c=path.charAt(i);
    if(c=='N') y++; else if(c=='S') y--;
    else if(c=='E') x++; else if(c=='W') x--;
}
return (float)Math.sqrt(x*x + y*y);
```

• *substring.java*

Concept: Extract a substring both manually (loop si..ei) and via built-in str.substring(a,b).

```
public static void print(String str,int si,int ei){
    for(int i=si;i&lt;ei;i++)
        System.out.print(str.charAt(i));
}
```

• *largeststring.java*

Concept: Lexicographically largest string: keep a running largest, replace when compareTo(other)<0.

```
String largest=str[0];
for(int i=0;i&lt;str.length;i++){
    if(largest.compareTo(str[i])&lt;0)
        largest=str[i];
}
```

• *stringbuilder.java*

Concept: Use StringBuilder.append() to build a string efficiently (mutable, avoids O(n^2) concatenation).

```
StringBuilder sb=new StringBuilder();
for(char ch='a';ch&lt;'k';ch++)
    sb.append(ch);
```

• *lowertouppercase.java*

Concept: Capitalize first letter of each word: uppercase char after a space using Character.toUpperCase.

```
char ch=Character.toUpperCase(str.charAt(0));
sb.append(ch);
for(int i=1;i&lt;str.length();i++){
    if(str.charAt(i)==' ' &amp;&amp; i&lt;str.length()-1){
        sb.append(str.charAt(i)); i++;
        sb.append(Character.toUpperCase(str.charAt(i)));
    } else sb.append(str.charAt(i));
}
```

• *compression.java*

Concept: Run-length encoding: count consecutive repeats, append char then count(if>1). e.g. aaabb -> a3b2.

```
for(int i=0;i&lt;str.length();i++){
    int count=1;
    while(i&lt;str.length()-1 &amp;&amp; str.charAt(i)==str.charAt(i+1)){
        count++; i++;
    }
    sb.append(str.charAt(i));
    if(count&gt;1) sb.append(count);
}
```

• *anagrams.java*

Concept: Frequency array int[26]: increment for str1, decrement for str2; all zero => anagram. O(n).

```
int arr[]=new int[26];
for(int i=0;i&lt;str1.length();i++) arr[str1.charAt(i)-'a']++;
for(int i=0;i&lt;str2.length();i++) arr[str2.charAt(i)-'a']--;
for(int i=0;i&lt;26;i++) if(arr[i]!=0) return false;
return true;
```

Bit Manipulation (14 solved)

Patterns covered: Core techniques: bitmask (1<<i) to get/set/clear/update a bit, AND with 1 for parity, range masks for clearing bit ranges, n&(n-1) power-of-two & set-bit tricks, binary exponentiation (fast/modular), XOR swap, and the ~ trick for +1.

Anchor problem: Get/Set/Clear i-th bit using a (1<<i) bitmask

• *basicbit.java*

Concept: Shift operators: a<<b = a*2^b (left), a>>b = a/2^b (right).

```
System.out.println(6&gt;&gt;1);   // 3
```

• *evenorodd.java*

Concept: Check LSB with bitmask 1: (1 & n)==0 means even.

```
int bitmask=1;
if((bitmask &amp; n)==0) System.out.println("even");
else System.out.println("odd");
```

• *getithbit.java*

Concept: Build mask 1<<i; if (mask & n)==0 the i-th bit is 0 else 1.

```
int bitmask=1&lt;&lt;i;
if((bitmask &amp; n)==0) System.out.println("0");
else System.out.println("1");
```

• *setithbit.java*

Concept: OR with 1<<i sets the i-th bit to 1.

```
int bitmask=1&lt;&lt;i;
System.out.println(n|bitmask);
```

• *clearithbit.java*

Concept: AND with ~(1<<i) clears (zeros) the i-th bit.

```
int bitmask=~(1&lt;&lt;i);
System.out.println(n&amp;bitmask);
```

• *updateithbit.java*

Concept: Place newbit at position i by shifting then OR (updateith uses newbit<<i).

```
int bitmask=newbit&lt;&lt;i;
System.out.println(n|bitmask);
```

• *clearlastibits.java*

Concept: Mask ~(0)<<i has zeros in the last i positions; AND clears the last i bits.

```
int bitmask=(~0)&lt;&lt;i;
System.out.println(n&amp;bitmask);
```

• *clearrangeofbits.java*

Concept: Clear bits i..j: a=(~0)<<(j+1) keeps high bits, b=(1<<i)-1 keeps low bits; mask=a|b.

```
int a=(~0)&lt;&lt;(j+1);
int b=(1&lt;&lt;i)-1;
int bitmask=a|b;
System.out.println(n&amp;bitmask);
```

• *nopowerof2nornot.java*

Concept: n is a power of 2 iff n&(n-1)==0 (only one set bit).

```
return (n&amp;(n-1))==0;
```

• *countsetbitinno.java*

Concept: Count 1s: check LSB with n&1, right-shift until 0.

```
int count=0;
while(n&gt;0){
    if((n&amp;1)!=0) count++;
    n=n&gt;&gt;1;
}
return count;
```

• *fastexponentiation.java*

Concept: Binary exponentiation: multiply into ans when current bit set, square base, shift exponent. O(log n).

```
int ans=1;
while(n&gt;0){
    if((n&amp;1)!=0) ans=ans*a;
    a=a*a;
    n=n&gt;&gt;1;
}
return ans;
```

• *modularexponentiation.java*

Concept: Fast exponentiation under modulo m: reduce base, multiply/square with % m each step. O(log b).

```
long result=1; a=a%m;
while(b&gt;0){
    if((b&amp;1)==1) result=(result*a)%m;
    a=(a*a)%m;
    b=b&gt;&gt;1;
}
return result;
```

• *add1toaninteger.java*

Concept: Add 1 without '+': n = -(~n), since ~n = -n-1.

```
n = -(~n);
```

• *swapthenowithout3rdvariable.java*

Concept: XOR swap: a^=b; b^=a; a^=b swaps without a temp variable.

```
a=a^b;
b=a^b;
a=a^b;
```

Divide and Conquer (3 solved)

Patterns covered: Core technique: split the problem in half, solve recursively, combine. Quick sort (partition around pivot), merge sort (sort halves then merge), and modified binary search on a rotated sorted array (decide which half is sorted).

Anchor problem: Merge Sort - divide, recursively sort halves, merge (O(n log n))

• *quicksort.java*

Concept: Partition around a pivot (last element): elements < pivot moved left, pivot placed at its index, recurse on both sides. Avg O(n log n).

```
int pidx=partition(arr,si,ei);
quicksorting(arr,si,pidx-1);
quicksorting(arr,pidx+1,ei);
// partition:
int pivot=arr[ei], i=si-1;
for(int j=si;j&lt;=ei;j++)
    if(arr[j]&lt;pivot){ i++; swap(arr,i,j); }
i++; swap(arr,i,ei);
return i;
```

• *mergesort.java*

Concept: Divide array at mid, recursively sort left & right, then merge the two sorted halves via a temp array. Stable, O(n log n), O(n) space.

```
int mid=si+(ei-si)/2;
mergesorting(arr,si,mid);
mergesorting(arr,mid+1,ei);
merge(arr,si,mid,ei);
// merge: pick smaller of arr[i],arr[j] into temp,
// copy leftovers, then copy temp back.
```

• *rotatedarray.java*

Concept: Search in a rotated sorted array: at each mid decide which half is sorted, then check if target lies in it to recurse. O(log n).

```
int mid=si+(ei-si)/2;
if(arr[mid]==target) return mid;
if(arr[si]&lt;=arr[mid]){            // left sorted
    if(target&gt;=arr[si] &amp;&amp; target&lt;=arr[mid])
        return search(arr,target,si,mid-1);
    else return search(arr,target,mid+1,ei);
} else {                          // right sorted
    if(target&gt;=arr[mid] &amp;&amp; target&lt;=arr[ei])
        return search(arr,target,mid+1,ei);
    else return search(arr,target,si,mid-1);
}
```

Matrix (7 solved)

Patterns covered: Core techniques: 2D traversal (row/col nested loops), diagonal indexing, boundary-shrinking spiral traversal, staircase search on a row/col-sorted matrix (O(n+m)), transpose (swap i,j), and marker-array approach for set-matrix-zeroes.

Anchor problem: Staircase Search - O(n+m) search in a sorted matrix

• *basicmatrix.java*

Concept: Read a matrix and find the smallest element with its (row,col) via full traversal.

```
for(int i=0;i&lt;matrix.length;i++)
    for(int j=0;j&lt;matrix[0].length;j++)
        if(matrix[i][j]&lt;smallest){
            smallest=matrix[i][j]; maxrow=i; maxcol=j;
        }
```

• *diagonalsumdsa.java*

Concept: Sum both diagonals in one pass: add matrix[i][i]; add matrix[i][n-i-1] unless it's the shared center.

```
for(int i=0;i&lt;n;i++){
    sum += matrix[i][i];
    if(i != n-i-1) sum += matrix[i][n-i-1];
}
```

• *spiralmatrixdsa.java*

Concept: Print in spiral by shrinking boundaries: top row -> right col -> bottom row -> left col, then move boundaries inward.

```
while(startrow&lt;=endrow &amp;&amp; startcol&lt;=endcol){
    for(int i=startcol;i&lt;=endcol;i++) print(matrix[startrow][i]);
    for(int j=startrow+1;j&lt;=endrow;j++) print(matrix[j][endcol]);
    if(startrow&lt;endrow)
      for(int i=endcol-1;i&gt;=startcol;i--) print(matrix[endrow][i]);
    if(startcol&lt;endcol)
      for(int j=endrow-1;j&gt;=startrow+1;j--) print(matrix[j][startcol]);
    startrow++; endrow--; startcol++; endcol--;
}
```

• *searchinsortedarray.java*

Concept: Staircase search on row+col sorted matrix: start top-right; move left if too big, down if too small. O(n+m).

```
int row=0,col=matrix[0].length-1;
while(row&lt;matrix.length &amp;&amp; col&gt;=0){
    if(key==matrix[row][col]) return true;
    else if(matrix[row][col]&gt;key) col--;
    else row++;
}
```

• *sumofrow.java*

Concept: Sum all elements of a specific row (here the 2nd row) by iterating its columns.

```
for(int j=0;j&lt;=matrix[0].length-1;j++)
    sum += matrix[1][j];
```

• *transposeofmatrix.java*

Concept: Transpose: transpose[j][i] = a[i][j]; result dimensions are cols x rows.

```
for(int i=0;i&lt;a.length;i++)
    for(int j=0;j&lt;a[0].length;j++)
        transpose[j][i]=a[i][j];
```

• *settozeromatrix.java*

Concept: Set-matrix-zeroes: first pass marks rows/cols containing a 0 in helper arrays; second pass zeroes any marked cell.

```
for(int i=0;i&lt;m;i++)
  for(int j=0;j&lt;n;j++)
    if(arr[i][j]==0){ colt[i]=1; rowt[j]=1; }
for(int i=0;i&lt;m;i++)
  for(int j=0;j&lt;n;j++)
    if(colt[i]==1 || rowt[j]==1) arr[i][j]=0;
```

OOP (11 solved)

Patterns covered: Core concepts: classes & objects, the four pillars (Encapsulation, Inheritance, Polymorphism, Abstraction), access modifiers (private/default/protected/public), getters/setters, constructors (default/parameterized/copy, shallow vs deep), the static keyword (class-shared members), and Java's GC-based destruction (no C++ destructor; finalize()).

Anchor problem: The 4 pillars + constructors (shallow vs deep copy)

• *classandobject.java*

Concept: A class groups attributes + methods; objects are instances. Set/read fields through the object reference.

```
pen p1 = new pen();
p1.setcolor("red");
p1.settip(5);
class pen{ String color; int tip;
    void setcolor(String c){ color=c; }
    void settip(int t){ tip=t; } }
```

• *accessmodifiers.java*

Concept: Access modifiers control visibility: private(class) < default(package) < protected(package+subclass) < public(everywhere). Class level allows only public/default.

```
class Account {
    private double balance;      // hidden
    public void deposit(double amt){
        if(amt&gt;0) balance+=amt;  // validated access
    }
}
```

• *gettersandsetters.java*

Concept: Encapsulation: keep fields private, expose read via getX() and controlled write via setX() (with validation).

```
private int age;
public int getAge(){ return age; }
public void setAge(int age){
    if(age&gt;0) this.age=age;
    else System.out.println("Invalid Age");
}
```

• *constructor.java*

Concept: Constructor initializes objects (same name, no return type). Types: default, parameterized, copy. Shallow copy shares references; deep copy duplicates nested objects.

```
Student(){ age=18; addr=new Address("Delhi"); }      // default
Student(int a,Address ad){ age=a; addr=ad; }          // parameterized
Student(Student s){ age=s.age; addr=s.addr; }         // shallow
Student deepCopy(){                                    // deep
    return new Student(age, new Address(addr.city));
}
```

• *destructors.java*

Concept: Java has no destructor; the Garbage Collector reclaims unreferenced objects. finalize() runs before GC (not guaranteed).

```
Student s = new Student();
s = null;        // eligible for GC
System.gc();     // request collection
protected void finalize(){ /* release resources */ }
```

• *inheritance.java*

Concept: extends lets a child class reuse a parent's members. Here fish inherits eat() from animal.

```
class animal{ void eat(){ System.out.println("fish"); } }
class fish extends animal { }
fish tuna = new fish();
tuna.eat();   // inherited
```

• *typesofinheritance.java*

Concept: Types: single, multilevel (chain), hierarchical (many children one parent). Multiple/hybrid via classes not allowed in Java (use interfaces).

```
class Animal{ void eat(){...} }
class Dog extends Animal{...}        // single
class Puppy extends Dog{...}         // multilevel
class Cat extends Animal{...}        // hierarchical
```

• *polymorphism.java*

Concept: Overriding (runtime): child redefines parent method. Overloading (compile-time): same name, different parameters.

```
Animal a = new Dog();
a.sound();          // Dog barks -&gt; overriding
m.add(2,3);         // overloading
m.add(2,3,4);
```

• *abstraction.java*

Concept: Hide implementation, show essentials. Achieved via abstract class (partial, abstract+concrete methods) or interface (full abstraction, implements).

```
abstract class Vehicle{
    abstract void start();
    void fuelType(){ System.out.println("Petrol/Diesel"); }
}
class Car extends Vehicle{
    void start(){ System.out.println("Car starts with key"); }
}
```

• *encapsulation.java*

Concept: Wrap data + methods in a class and restrict direct access: private fields + getter/setter = data hiding.

```
class Student{
    private int age;              // data hiding
    public int getAge(){ return age; }
    public void setAge(int age){ if(age&gt;0) this.age=age; }
}
```

• *statickeyword.java*

Concept: static members belong to the class, shared by all objects (e.g., a common schoolName). Changing it via any instance affects all.

```
class student{
    static String schoolname;   // shared across objects
}
s1.schoolname="hsa";
s2.schoolname="...";  // both refer to same static field
```

Recursion (14 solved)

Patterns covered: Core techniques: base case + recursive call, head vs tail recursion (print order), linear recursion for factorial/sum/fibonacci, index-driven array recursion (first/last occurrence, sorted check), fast power (halve exponent), and choice-based counting/backtracking (tiling, friends-pairing, binary strings, remove duplicates).

Anchor problem: Fast power x^n by halving the exponent (O(log n))

• *printno1ton.java*

Concept: Increasing print: recurse first (n-1) then print n -> prints 1..n (work after recursion).

```
public static void printinc(int n){
    if(n==1){ System.out.println(n); return; }
    printinc(n-1);
    System.out.println(n);
}
```

• *decreasingorderofnum.java*

Concept: Decreasing print: print n first then recurse -> prints n..1 (work before recursion).

```
public static void printdec(int n){
    if(n==1){ System.out.println(n); return; }
    System.out.println(n);
    printdec(n-1);
}
```

• *factorialofn.java*

Concept: n! = n * (n-1)!; base case 0!=1.

```
public static int fact(int n){
    if(n==0) return 1;
    return n * fact(n-1);
}
```

• *sumoffirstnaturalno.java*

Concept: sum(n) = n + sum(n-1); base case sum(1)=1.

```
public static int sum(int n){
    if(n==1) return 1;
    return n + sum(n-1);
}
```

• *printnthfibonaacino.java*

Concept: fib(n)=fib(n-1)+fib(n-2); base cases fib(0)=0, fib(1)=1. (Exponential tree recursion.)

```
public static int fibonacci(int n){
    if(n==0) return 0;
    if(n==1) return 1;
    return fibonacci(n-1)+fibonacci(n-2);
}
```

• *firstoccurenceofelementinarray.java*

Concept: Scan left-to-right via index i; return i on first match, recurse otherwise.

```
public static int firstocc(int arr[],int i,int key){
    if(i==arr.length) return -1;
    if(arr[i]==key) return i;
    return firstocc(arr,i+1,key);
}
```

• *lastoccurenceofelemetinarray.java*

Concept: Recurse to the end first; on the way back, return i only if a later occurrence wasn't found.

```
public static int lastocc(int arr[],int i,int key){
    if(i==arr.length) return -1;
    int isfound=lastocc(arr,i+1,key);
    if(isfound==-1 &amp;&amp; arr[i]==key) return i;
    return isfound;
}
```

• *printxtothepowern.java*

Concept: Fast power: halfpow=pow(x,n/2); square it; multiply extra x if n is odd. O(log n).

```
public static int optimisedpower(int x,int n){
    if(n==0) return 1;
    int half=optimisedpower(x,n/2);
    int sq=half*half;
    if(n%2!=0) sq=x*sq;
    return sq;
}
```

• *optimisedpowerxton.java*

Concept: Notes/derivation: x^n = x^(n/2)*x^(n/2), and for odd n multiply an extra x (companion to fast power).

```
/* n even: x^n = x^(n/2) * x^(n/2)
   n odd : x^n = x * x^(n/2) * x^(n/2) */
```

• *dsatillinproblem.java*

Concept: Tiling 2xN floor with 2x1 tiles: place vertical -> tiling(n-1), or horizontal pair -> tiling(n-2). ways(n)=ways(n-1)+ways(n-2).

```
public static int tiling(int n){
    if(n==0 || n==1) return 1;
    return tiling(n-1) + tiling(n-2);
}
```

• *removeduplicatesinstring.java*

Concept: Backtracking with a seen[] map: skip char if already used, else mark and append. Builds a deduped string.

```
char ch=str.charAt(idx);
if(map[ch-'a']) removedup(str,idx+1,newstr,map);
else { map[ch-'a']=true;
       removedup(str,idx+1,newstr.append(ch),map); }
```

• *friendspair.java*

Concept: Each friend stays single -> f(n-1), or pairs with one of (n-1) others -> (n-1)*f(n-2). total=f(n-1)+(n-1)*f(n-2).

```
public static int friendpair(int n){
    if(n==1 || n==2) return n;
    return friendpair(n-1) + (n-1)*friendpair(n-2);
}
```

• *arrayissortedornot.java*

Concept: Check sorted via index: false if arr[i]>arr[i+1], else recurse; true at last index.

```
public static boolean sortedornot(int arr[],int i){
    if(i==arr.length-1) return true;
    if(arr[i]&gt;arr[i+1]) return false;
    return sortedornot(arr,i+1);
}
```

• *binarystring.java*

Concept: Generate binary strings of length n with no consecutive 1s: always place 0; place 1 only if last placed was 0.

```
if(n==0){ System.out.println(str); return; }
printBinaryString(n-1,0,str+"0");
if(lastPlaced==0)
    printBinaryString(n-1,1,str+"1");
```

Sorting (6 solved)

Patterns covered: Core techniques: comparison sorts - bubble (swap adjacent), selection (pick min each pass), insertion (shift into sorted prefix), all O(n^2); counting sort (non-comparison, O(n+k)); and Java's built-in Arrays.sort / Collections.reverseOrder.

Anchor problem: Insertion Sort - shift each element into its sorted position

• *bubblesort.java*

Concept: Repeatedly swap adjacent out-of-order elements; largest 'bubbles' to the end each pass. O(n^2), O(1).

```
for(int i=0;i&lt;arr.length-1;i++)
    for(int j=0;j&lt;arr.length-1-i;j++)
        if(arr[j]&gt;arr[j+1]){
            int t=arr[j]; arr[j]=arr[j+1]; arr[j+1]=t;
        }
```

• *selectionsort.java*

Concept: Find the minimum in the unsorted part and swap it to the front each pass. O(n^2).

```
for(int i=0;i&lt;arr.length-1;i++){
    int min=i;
    for(int j=i+1;j&lt;arr.length;j++)
        if(arr[j]&lt;arr[min]) min=j;
    int t=arr[min]; arr[min]=arr[i]; arr[i]=t;
}
```

• *insertionsort.java*

Concept: Pick each element and shift larger sorted elements right to insert it at the correct spot. O(n^2), good for nearly-sorted.

```
for(int i=0;i&lt;arr.length-1;i++){
    int current=arr[i+1], j=i;
    while(j&gt;=0 &amp;&amp; arr[j]&gt;current){
        arr[j+1]=arr[j]; j--;
    }
    arr[j+1]=current;
}
```

• *countingsort.java*

Concept: Non-comparison sort: count occurrences into count[], then rebuild array in order. O(n+k), good for small integer ranges.

```
int countarr[]=new int[max+1];
for(int i=0;i&lt;arr.length;i++) countarr[arr[i]]++;
int j=0;
for(int i=0;i&lt;countarr.length;i++)
    while(countarr[i]--&gt;0) arr[j++]=i;
```

• *javainbuiltsort.java*

Concept: Java's Arrays.sort(arr, from, to) sorts a range; uses dual-pivot quicksort for primitives.

```
Arrays.sort(arr,0,4);   // sorts indices 0..3
```

• *inbuiltcollection.java*

Concept: Note: Collections.reverseOrder() with Arrays.sort sorts in descending order (works on Integer[] objects).

```
// Arrays.sort(arr, Collections.reverseOrder());
```

Practice (1 solved)

Patterns covered: Mixed revision folder. Currently reinforces Kadane's algorithm (max subarray sum).

Anchor problem: Kadane's Algorithm (revision)

• *practice.java*

Concept: Revision of Kadane's algorithm: running sum, track max, reset when sum goes negative. O(n).

```
int currsum=0, maxsum=Integer.MIN_VALUE;
for(int i=0;i&lt;arr.length;i++){
    currsum+=arr[i];
    maxsum=Math.max(maxsum,currsum);
    if(currsum&lt;0) currsum=0;
}
```